4x4 Tic-Tac-Toe Architecture Breakdown

https://gemini.google.com/app/2dcd2474539ccab8

User prompt: I need to create a 4x4 solver which looks to see if a board has a valid win in Vertical, Horizontal, Diagonal, All Four Corner, 2x2 box. I need to check if any moves still exist, if the game is over, and who has won a current game state. Don't implement, just help me understand how to use this hypothetical class better. Do you think it should store an instance of the game? Should it be used statically and just always passed in the current game state? Is an array of strings the best way to represent the game state? Might a function to translate a string into a game state be useful (like XX-OO-XO-XXOXXOX) represent the 16 characters with - being blank?

Response: State Ownership: Static Utility vs. Stateful Instance

A **pure, stateless evaluator** (used statically or as a singleton service) is typically much cleaner than binding the solver to an internal board instance.

- **Stateless Solver (`BoardEvaluator` / `Solver`):** Accept the board state as an argument and return an immutable evaluation result (e.g., `EvaluationResult { isGameOver, winner, hasMovesLeft }`).
    - **Why it shines:** Game loops, minimax trees, testing harnesses, and unit tests don't have to instantiate or reset an object every time they inspect a move. You pass in a state, get an answer, and move on.
- **When a Stateful Class Makes Sense:** If this class isn't just an evaluator, but represents the **actual game engine** (managing turns, validating move inputs, undo/redo history, and maintaining match status), then it should own its state. Even then, separating the *Game Model* from the *Rule/Win Evaluator* keeps your code decoupled and easy to test.

Board Representation: Array of Strings vs. Alternatives

An array of strings (e.g., `["X", "O", "-", "X", ...]`) works fine for small prototypes, but it has trade-offs:

Representation	Pros	Cons / Verdict
Array of Strings / Chars (`char[16]` or `string[16]`)	Highly readable, trivial to print, direct mapping to board indexes (0 to 15).	String allocations and string-equality checks are slower than primitive checks (though negligible at human speeds, noticeable if writing AI search algorithms).
Enum / Byte Array (`enum Cell { EMPTY, X, O }`)	Strongly typed, prevents invalid character bugs, low memory footprint.	**Recommended standard choice** for readability, maintainability, and clean object modeling.
Bitboards (`uint16` / `ushort` per player)	Blazing fast. A 4×4 grid has exactly 16 cells, fitting in a 16-bit integer.	Win checks become instant bitwise AND masks. Less intuitive to debug at a glance, but ideal for Monte Carlo or Minimax engines.

String Serialization & Parsing (XX–00–X0–XX0XX0X)

Yes, implementing a serializer/deserializer to convert a 16-character string into your internal state is **exceptionally useful**, particularly for:

- **Unit Testing:** You can write concise, readable test cases in a single line without tedious multi-line array setups:

  `solver.evaluate(Board.fromString("XXXX------------")) -> Winner: X`
- **Debugging & Logging:** Dumping board states directly into bug reports, terminal outputs, or log files.
- **Saving & Networking:** Easy persistence (saving a match to disk, sending state over an API/socket, or encoding in a URL query parameter).

Modeling the 4x4 Win Conditions

A 4×4 grid has several distinct winning patterns to cover:

- **Rows (Horizontal):** 4 lines of 4
- **Columns (Vertical):** 4 lines of 4
- **Diagonals:** 2 lines of 4 (main and anti-diagonal)
- **Four Corners:** 1 pattern (indices `0, 3, 12, 15`)
- 2×2 **Sub-boxes:** 9 possible boxes across the board (3 horizontal offsets $\times 3$ vertical offsets)

Total winning configurations: **20 distinct patterns**.

By predefining these 20 winning combinations as sets of 4 indices (e.g., an array of index tuples or 16-bit masks), your evaluation logic remains a simple loop over the patterns, checking if any single player occupies all 4 indices in any set.

Suggested API Contract

A clean design separates the evaluation output from the input board:

```
Class: BoardEvaluator (Stateless)

Methods:
  - evaluate(board: BoardState) -> GameStatus
```

```
      Returns:
        * isGameOver: boolean
        * winner: Player | null
        * hasMovesLeft: boolean
        * winningLine: WinPattern | null (useful for UI highlight overlays)

Class: BoardState (or GameState)
  - fromString(stateString: string) -> BoardState
  - toString() -> string
  - getCell(row, col) / getCell(index)
  - isFull() -> boolean
```

User prompt: should my parse board function expect a well formatted string, or should it check string length and make sure it only recieves "O" "X" and dash or space?

Response: Your parse function should **strictly validate the string** rather than assuming well-formatted input.

In application architecture, parsing functions serve as the boundary gatekeeper. Validating at the parser level protects the rest of your system (evaluators, minimax algorithms, UI renderers) from having to perform defensive checks downstream.

Why Strict Validation Is Preferable

- **Fail Fast, Fail Loud:** If a typo occurs (e.g., passing a 15-character string or accidental lowercase `'o'`), catching it inside `parseBoard()` with an informative error immediately identifies the bug at the source. Otherwise, silent corruption leads to downstream index-out-of-bounds errors or incorrect win evaluations.
- **Simplifies Downstream Logic:** Once the parser returns a valid internal representation (like an `Enum[]` or a `Bitboard`), every other class—including your evaluator and AI solver—can safely assume the board is 100% structurally sound.
- **Separation of Concerns:**
  - The **parser** validates syntax (length, permitted characters).
  - The **evaluator** validates rules (who won, board full).
  - An optional **game validator** checks game legality (e.g., difference between X count and 0 count cannot exceed 1).

What the Parser Should Validate

1. **Length:** Exactly 16 characters (or length 16 after stripping optional formatting whitespace/newlines).
2. **Character Whitelist:** Only accept `'X'`, `'O'`, and your designated empty marker (e.g., `'-'` or `' '`).
3. **Normalization (Optional Convenience):** Deciding whether to auto-convert lowercase x/o to uppercase, and whether to treat both spaces and hyphens as blanks, makes testing and copy-pasting easier without compromising integrity.

Recommended Validation Design Patterns

Depending on how your codebase handles failures, two patterns work best:

1. Throwing Factory (`Board.fromString(str)`)

Ideal if invalid input represents a programmer error or broken invariant (e.g., in unit tests):

```
Board.fromString(str):
  1. Check if str is null/undefined -> throw InvalidArgumentError
  2. If str.length != 16 -> throw InvalidArgumentError("Board string must be 16 characters, got " + str.length)
  3. For each char at index i:
       - If char is 'X' -> set Cell.X
       - If char is 'O' -> set Cell.O
       - If char is '-' or ' ' -> set Cell.EMPTY
       - Else -> throw InvalidArgumentError("Unrecognized token '" + char + "' at position " + i)
  4. Return valid Board instance
```

2. Result / `TryParse` Pattern (`Board.tryParse(str)`)

Ideal if strings come from external or untrusted sources (e.g., query params, user text inputs, network packets):

```
Board.tryParse(str) -> Result<Board, ParseError>
  - Returns a success object containing the Board, OR
  - Returns a failure object containing the specific reason (e.g., "INVALID_LENGTH", "INVALID_CHAR").
```

User prompt: // Implement a 4x4 Tic-Tac-Toe/Connect Four solver. The program should take in a representation of a 4x4 board, // and tell you the winner if any. // Victory is determined by one of five conditions: // Vertical // Horizontal // Diagonal // All Four Corner // 2x2 box // Implementing a UI is not necessary: we are only interested in your implementation of the win condition. // Implement the below functions, and feel free to create any other types that you need, // as well as unit tests (just tell us how to run them!). //If you use AI please include your AI session transcripts. type Player = "X" | "O"; type

Cell = Player | null; type Board = Cell[]; // Length 16 for 4x4 type CellTuple = [number, number, number, number]; // Represents a 4-tuple of cell indices for win conditions type GameResult = "In Progress" | Player | "Draw"; const WINNING_COMBINATIONS: CellTuple[] = [//there are only so many win conditions for 4x4, so hardcode them // horizontal (4 rows) [0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11], [12, 13, 14, 15], // vertical (4 cols) [0, 4, 8, 12], [1, 5, 9, 13], [2, 6, 10, 14], [3, 7, 11, 15], // main diags (2) [0, 5, 10, 15], [3, 6, 9, 12], // corners [0, 3, 12, 15], // 2x2 boxes (9 sub-grids) [0, 1, 4, 5], [1, 2, 5, 6], [2, 3, 6, 7], [4, 5, 8, 9], [5, 6, 9, 10], [6, 7, 10, 11], [8, 9, 12, 13], [9, 10, 13, 14], [10, 11, 14, 15],]; class TicTacToe { public static parseBoard(boardStr: string): Board { if (boardStr.length !== 16) { throw new Error("Invalid board length. Expected 16 characters."); } return boardStr.split("").map((char, index) => { switch (char) { case "X": return "X"; case "O": return "O"; case "-": case " ": return null; default: throw new Error(`Invalid character '${char}'`); } }); } public static checkWinner(board: Board): Player | null { for (const combination of WINNING_COMBINATIONS) { const [a, b, c, d] = combination; if (board[a] !== null && board[a] === board[b] && board[a] === board[c] && board[a] === board[d]) { return board[a]; } } return null; } public static anyMovesLeft(board: Board): boolean { return board.some((cell) => cell === null); } public static isGameOver(board: Board): GameResult { const winner = this.checkWinner(board); if (winner) { return winner; } if (!this.anyMovesLeft(board)) { return "Draw"; } return "In Progress"; } } can you help me write test cases for this? I want them in the same file as main, ideally no need to import any assertion classes or anythign

Response: Here is a zero-dependency test runner with unit tests for each win condition, draw states, in-progress states, and board parser error validation.

You can append this directly to the bottom of your file.

```
// ==========================================
// Minimal Zero-Dependency Test Framework
// ==========================================

function assertEqual<T>(actual: T, expected: T, testName: string): void {
  if (actual !== expected) {
    console.error(`❌ FAILED: ${testName}`);
    console.error(`   Expected: ${JSON.stringify(expected)}`);
    console.error(`   Actual:   ${JSON.stringify(actual)}`);
  } else {
    console.log(`✅ PASSED: ${testName}`);
  }
}

function assertThrows(fn: () => void, testName: string): void {
  try {
    fn();
    console.error(`❌ FAILED: ${testName} (Expected function to throw, but it did not)`);
  } catch {
    console.log(`✅ PASSED: ${testName}`);
  }
}

// ==========================================
// Test Suite
// ==========================================

function runTests(): void {
  console.log("=== Running 4x4 Solver Tests ===\n");

  // 1. Horizontal Wins (Rows 1 & 4)
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("XXXX------------")),
    "X",
    "Horizontal Win (Row 0 for X)"
  );
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("------------OOOO")),
    "O",
    "Horizontal Win (Row 3 for O)"
  );

  // 2. Vertical Wins (Columns 1 & 4)
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("X---X---X---X---")),
    "X",
    "Vertical Win (Col 0 for X)"
  );
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("---O---O---O---O")),
    "O",
    "Vertical Win (Col 3 for O)"
  );

  // 3. Diagonal Wins (Main & Anti-diagonal)
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("X----X----X----X")),
    "X",
    "Main Diagonal Win (\)"
  );
```

```
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("---O--O--O--O---")),
    "O",
    "Anti-Diagonal Win (/)"
  );

  // 4. All Four Corners Win (Indices 0, 3, 12, 15)
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("X--X--------X--X")),
    "X",
    "Four Corners Win"
  );

  // 5. 2x2 Box Wins (Top-Left and Center)
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("OO--OO----------")),
    "O",
    "2x2 Sub-grid Win (Top-Left: 0, 1, 4, 5)"
  );
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("-----XX---XX----")),
    "X",
    "2x2 Sub-grid Win (Center: 5, 6, 9, 10)"
  );

  // 6. Game State Progression
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("----------------")),
    "In Progress",
    "Empty Board Status"
  );
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("X O X O - - - - ")),
    "In Progress",
    "Partial Board with Spaces (In Progress)"
  );
  assertEqual(
    TicTacToe.anyMovesLeft(TicTacToe.parseBoard("XOXOXOXOXOXOXOXO")),
    false,
    "No Moves Left Check"
  );
  assertEqual(
    TicTacToe.isGameOver(TicTacToe.parseBoard("XOXOOXOXOXOXOXOX")),
    "Draw",
    "Full Board Draw (No Win Condition Met)"
  );

  // 7. Parser Input Validation
  assertThrows(
    () => TicTacToe.parseBoard("XX--OO"),
    "Parser Error on Short String (Length 6)"
  );
  assertThrows(
    () => TicTacToe.parseBoard("XXXXXXXXXXXXXXXXX"),
    "Parser Error on Long String (Length 17)"
  );
  assertThrows(
    () => TicTacToe.parseBoard("XXXX----????----"),
    "Parser Error on Invalid Tokens (?)"
  );

  console.log("\n=== Test Suite Complete ===");
}

// Execute tests
runTests();
```

How to Run

Save your entire file (e.g. `solver.ts`) and run it using any TypeScript-capable runtime:

Using `ts-node`:

```
npx ts-node solver.ts
```

Using `tsx` (fastest):

```
npx tsx solver.ts
```

Using the TypeScript compiler (`tsc`):

```
npx tsc solver.ts
node solver.js
```

User prompt: class Main { public static main(): void { const winnerTestCases: TestCase[] = [// horizontal { board: "XXXX----------", expected: "X", desc: "Horizontal Row 0" }, { board: "----OOOO--------", expected: "O", desc: "Horizontal Row 1" }, { board: "----------XXXX", expected: "X", desc: "Horizontal Row 3" }, // vertical { board: "X---X---X---X---", expected: "X", desc: "Vertical Col 0" }, { board: "---O---O---O---O", expected: "O", desc: "Vertical Col 3" }, // diagonal { board: "X----X----X----X", expected: "X", desc: "Main Diagonal" }, { board: "---O--O--O--O---", expected: "O", desc: "Anti-Diagonal" }, // corners { board: "X--X--------X--X", expected: "X", desc: "All 4 Corners" }, // 2x2 box { board: "XX--XX----------", expected: "X", desc: "Top-Left 2x2 Box" }, { board: "-----OO--OO-----", expected: "O", desc: "Center 2x2 Box" }, { board: "----------XX--XX", expected: "X", desc: "Bottom-Right 2x2 Box", }, // in progress or draw { board: "----------------", expected: "In Progress", desc: "Empty Board", }, { board: "X-O-X-O---------", expected: "In Progress", desc: "Mid Game with moves left", }, { board: "XOXOOXOXOXOXOXOX", expected: "Draw", desc: "Full Board Draw" }, { board: "-XX-XX-XX-XX-XX-", expected: "In Progress", desc: "Almost Win", },]; const MovesLeftTestCases: TestCase[] = [{ board: "----------------", expected: true, desc: "Empty Board" }, { board: "X-O-X-O---------", expected: true, desc: "Mid Game with moves left", }, { board: "XOXOOXOXOXOXOXOX", expected: false, desc: "Full Board Draw" }, { board: "XXXXXXXXXXXXXXXX", expected: false, desc: "Full Board Win" },]; let passedCount = 0; for (const test of winnerTestCases) { const board = TicTacToe.parseBoard(test.board); const actual = TicTacToe.isGameOver(board); if (actual === test.expected) { passedCount++; } else { console.error(`FAIL: ${test.desc}\n Board: ${test.board}\n Expected: ${test.expected}\n Actual: ${actual}`,); } } console.log(`\nTests Completed: ${passedCount}/${testCases.length} Passed`); } } Main.main(); how do i make the test cases run on both sets using the appropriate function?

Response: To run both sets cleanly, define a generic `TestCase<T>` type and extract the execution loop into a reusable helper method that accepts the test array and a function to run.

Here is the updated `Main` class:

```
// 1. Generic TestCase supporting any expected return type
interface TestCase<T> {
  board: string;
  expected: T;
  desc: string;
}

class Main {
  // 2. Reusable test runner helper
  private static runSuite<T>(
    suiteName: string,
    cases: TestCase<T>[],
    fn: (board: Board) => T
  ): void {
    console.log(`\n=== Running ${suiteName} ===`);
    let passedCount = 0;

    for (const test of cases) {
      const board = TicTacToe.parseBoard(test.board);
      const actual = fn(board);

      if (actual === test.expected) {
        passedCount++;
      } else {
        console.error(
          `FAIL: ${test.desc}\n  Board:    ${test.board}\n  Expected: ${test.expected}\n  Actual:   ${actual}\n`
        );
      }
    }

    console.log(`${suiteName} Results: ${passedCount}/${cases.length} Passed`);
  }

  public static main(): void {
    const winnerTestCases: TestCase<GameResult>[] = [
      // horizontal
      { board: "XXXX----------", expected: "X", desc: "Horizontal Row 0" },
      { board: "----OOOO--------", expected: "O", desc: "Horizontal Row 1" },
      { board: "----------XXXX", expected: "X", desc: "Horizontal Row 3" },
      // vertical
      { board: "X---X---X---X---", expected: "X", desc: "Vertical Col 0" },
      { board: "---O---O---O---O", expected: "O", desc: "Vertical Col 3" },
      // diagonal
      { board: "X----X----X----X", expected: "X", desc: "Main Diagonal" },
      { board: "---O--O--O--O---", expected: "O", desc: "Anti-Diagonal" },
      // corners
      { board: "X--X--------X--X", expected: "X", desc: "All 4 Corners" },
      // 2x2 box
      { board: "XX--XX----------", expected: "X", desc: "Top-Left 2x2 Box" },
```

```
      { board: "-----OO--OO-----", expected: "O", desc: "Center 2x2 Box" },
      { board: "----------XX--XX", expected: "X", desc: "Bottom-Right 2x2 Box" },
      // in progress or draw
      { board: "----------------", expected: "In Progress", desc: "Empty Board" },
      { board: "X-O-X-O---------", expected: "In Progress", desc: "Mid Game with moves left" },
      { board: "XOXOOXOXOXOXOXOX", expected: "Draw", desc: "Full Board Draw" },
      { board: "-XX-XX-XX-XX-XX-", expected: "In Progress", desc: "Almost Win" },
    ];

    const movesLeftTestCases: TestCase<boolean>[] = [
      { board: "----------------", expected: true, desc: "Empty Board" },
      { board: "X-O-X-O---------", expected: true, desc: "Mid Game with moves left" },
      { board: "XOXOOXOXOXOXOXOX", expected: false, desc: "Full Board Draw" },
      { board: "XXXXXXXXXXXXXXXX", expected: false, desc: "Full Board Win" },
    ];

    // 3. Pass each test list and target method
    Main.runSuite("Game Status Tests", winnerTestCases, (board) => TicTacToe.isGameOver(board));
    Main.runSuite("Moves Left Tests", movesLeftTestCases, (board) => TicTacToe.anyMovesLeft(board));
  }
}

Main.main();
```

Key Changes

- **`TestCase<T>` Generic:** Allows `expected` to be typed as `GameResult` for one suite and `boolean` for the other without using `any`.
- **`runSuite<T>` Function:** Extracts the iteration, parsing, assertions, and reporting so you don't repeat the loop logic.
- **Arrow Callbacks:** Passing `(board) => TicTacToe.isGameOver(board)` and `(board) => TicTacToe.anyMovesLeft(board)` binds the correct function dynamically to the corresponding test cases.

User prompt: is there a simpler way to do it? just run the test loop twice and sum up the passed tests?

Response: Yes, you can just write a second `for` loop directly inside `main()` and accumulate `passedCount` across both.

Here is the straightforward version without helper methods or generics:

```
type TestCase = {
  board: string;
  expected: any;
  desc: string;
};

class Main {
  public static main(): void {
    const winnerTestCases: TestCase[] = [
      // horizontal
      { board: "XXXX------------", expected: "X", desc: "Horizontal Row 0" },
      { board: "----OOOO--------", expected: "O", desc: "Horizontal Row 1" },
      { board: "------------XXXX", expected: "X", desc: "Horizontal Row 3" },
      // vertical
      { board: "X---X---X---X---", expected: "X", desc: "Vertical Col 0" },
      { board: "---O---O---O---O", expected: "O", desc: "Vertical Col 3" },
      // diagonal
      { board: "X----X----X----X", expected: "X", desc: "Main Diagonal" },
      { board: "---O--O--O--O---", expected: "O", desc: "Anti-Diagonal" },
      // corners
      { board: "X--X--------X--X", expected: "X", desc: "All 4 Corners" },
      // 2x2 box
      { board: "XX--XX----------", expected: "X", desc: "Top-Left 2x2 Box" },
      { board: "-----OO--OO-----", expected: "O", desc: "Center 2x2 Box" },
      { board: "----------XX--XX", expected: "X", desc: "Bottom-Right 2x2 Box" },
      // in progress or draw
      { board: "----------------", expected: "In Progress", desc: "Empty Board" },
      { board: "X-O-X-O---------", expected: "In Progress", desc: "Mid Game with moves left" },
      { board: "XOXOOXOXOXOXOXOX", expected: "Draw", desc: "Full Board Draw" },
      { board: "-XX-XX-XX-XX-XX-", expected: "In Progress", desc: "Almost Win" },
    ];

    const movesLeftTestCases: TestCase[] = [
      { board: "----------------", expected: true, desc: "Empty Board" },
      { board: "X-O-X-O---------", expected: true, desc: "Mid Game with moves left" },
```

```
      { board: "XOXOOXOXOXOXOXOX", expected: false, desc: "Full Board Draw" },
      { board: "XXXXXXXXXXXXXXXX", expected: false, desc: "Full Board Win" },
    ];

    let passedCount = 0;
    const totalTests = winnerTestCases.length + movesLeftTestCases.length;

    // Loop 1: Game Status Tests
    for (const test of winnerTestCases) {
      const board = TicTacToe.parseBoard(test.board);
      const actual = TicTacToe.isGameOver(board);

      if (actual === test.expected) {
        passedCount++;
      } else {
        console.error(
          `FAIL: ${test.desc}\n  Board:    ${test.board}\n  Expected: ${test.expected}\n  Actual:   ${actual}`
        );
      }
    }

    // Loop 2: Moves Left Tests
    for (const test of movesLeftTestCases) {
      const board = TicTacToe.parseBoard(test.board);
      const actual = TicTacToe.anyMovesLeft(board);

      if (actual === test.expected) {
        passedCount++;
      } else {
        console.error(
          `FAIL: ${test.desc}\n  Board:    ${test.board}\n  Expected: ${test.expected}\n  Actual:   ${actual}`
        );
      }
    }

    console.log(`\nTests Completed: ${passedCount}/${totalTests} Passed`);
  }
}

Main.main();
```
